

Python Coding

Interview Practice Guide

60+ Easy to Medium Questions with Solutions

TABLE OF CONTENTS

1. Strings (15 questions)
2. Lists & Arrays (12 questions)
3. Dictionaries & Sets (10 questions)
4. Functions & Recursion (8 questions)
5. File I/O & Data Processing (8 questions)
6. Object-Oriented Programming (6 questions)
7. Common Algorithms (8 questions)
8. Interview Tips & Patterns

1. STRINGS (15 Questions)

Q1: Reverse a String

Problem: Write a function to reverse a string. **Example:** Input: "hello" Output: "olleh"

```
# Solution 1: Slicing (Most Pythonic) def reverse_string(s): return s[::-1] # Solution 2: Using reversed()
def reverse_string_v2(s): return ''.join(reversed(s)) # Solution 3: Manual loop def reverse_string_v3(s):
result = [] for i in range(len(s) - 1, -1, -1): result.append(s[i]) return ''.join(result) # Solution 4:
Using list and reverse def reverse_string_v4(s): chars = list(s) chars.reverse() return ''.join(chars) #
Test print(reverse_string("hello")) # "olleh" print(reverse_string("Python")) # "nohtyP"
```

Q2: Check if String is Palindrome

Problem: Check if a string is a palindrome (reads same forwards and backwards). Ignore case and spaces. **Example:**

Input: "A man a plan a canal Panama" Output: True

```
# Solution 1: Clean and compare def is_palindrome(s): # Remove spaces and convert to lowercase cleaned =
s.replace(" ", "").lower() return cleaned == cleaned[::-1] # Solution 2: Two pointers (more efficient) def
is_palindrome_v2(s): s = s.replace(" ", "").lower() left, right = 0, len(s) - 1 while left < right: if
s[left] != s[right]: return False left += 1 right -= 1 return True # Solution 3: Using filter for
alphanumeric only def is_palindrome_v3(s): # Keep only alphanumeric characters cleaned =
''.join(filter(str.isalnum, s)).lower() return cleaned == cleaned[::-1] # Test
print(is_palindrome("racecar")) # True print(is_palindrome("hello")) # False print(is_palindrome("A man a
plan a canal Panama")) # True
```

Q3: Count Vowels in String

Problem: Count the number of vowels (a, e, i, o, u) in a string. Case-insensitive. **Example:** Input: "Hello World" Output: 3 (e, o, o)

```
# Solution 1: Using count in loop
def count_vowels(s):
    vowels = "aeiouAEIOU"
    count = 0
    for char in s:
        if char in vowels:
            count += 1
    return count

# Solution 2: List comprehension
def count_vowels_v2(s):
    vowels = "aeiouAEIOU"
    return sum(1 for char in s if char in vowels)

# Solution 3: Using filter
def count_vowels_v3(s):
    vowels = set("aeiouAEIOU")
    return len(list(filter(lambda x: x in vowels, s)))

# Solution 4: Dictionary with counts
def count_vowels_detailed(s):
    vowel_count = {'a': 0, 'e': 0, 'i': 0, 'o': 0, 'u': 0}
    for char in s.lower():
        if char in vowel_count:
            vowel_count[char] += 1
    return vowel_count

# Test
print(count_vowels("Hello World")) # 3
print(count_vowels("Python")) # 1
print(count_vowels_detailed("Beautiful")) # {'a': 1, 'e': 1, 'i': 1, 'o': 0, 'u': 2}
```

Q4: First Non-Repeating Character

Problem: Find the first non-repeating character in a string. **Example:** Input: "leetcode" Output: "l" Input: "loveleetcode" Output: "v"

```
# Solution 1: Using Counter (Best for readability)
from collections import Counter
def first_unique_char(s):
    char_count = Counter(s)
    for char in s:
        if char_count[char] == 1:
            return char
    return None

# Solution 2: Manual counting with dict
def first_unique_char_v2(s):
    char_count = {}
    for char in s:
        char_count[char] = char_count.get(char, 0) + 1
    for char in s:
        if char_count[char] == 1:
            return char
    return None

# Solution 3: Using index and rindex (clever but less efficient)
def first_unique_char_v3(s):
    for i, char in enumerate(s):
        if s.index(char) == s.rindex(char):
            return char
    return None

# Test
print(first_unique_char("leetcode")) # "l"
print(first_unique_char("loveleetcode")) # "v"
print(first_unique_char("aabb")) # None
```

Q5: Anagram Check

Problem: Check if two strings are anagrams (contain same characters with same frequency). **Example:** Input: "listen", "silent" Output: True Input: "hello", "world" Output: False

```
# Solution 1: Using sorted (Simplest) def is_anagram(s1, s2): return sorted(s1) == sorted(s2) # Solution
2: Using Counter (More explicit) from collections import Counter def is_anagram_v2(s1, s2): return
Counter(s1) == Counter(s2) # Solution 3: Manual character count def is_anagram_v3(s1, s2): if len(s1) !=
len(s2): return False char_count = {} for char in s1: char_count[char] = char_count.get(char, 0) + 1 for
char in s2: if char not in char_count: return False char_count[char] -= 1 if char_count[char] < 0: return
False return all(count == 0 for count in char_count.values()) # Test print(is_anagram("listen", "silent"))
# True print(is_anagram("hello", "world")) # False print(is_anagram("anagram", "nagaram")) # True
```

Q6: Remove Duplicates from String

Problem: Remove duplicate characters from a string while maintaining order. **Example:** Input: "programming" Output: "progamin"

```
# Solution 1: Using set to track seen characters def remove_duplicates(s): seen = set() result = [] for
char in s: if char not in seen: seen.add(char) result.append(char) return ''.join(result) # Solution 2:
Using dict.fromkeys (preserves order in Python 3.7+) def remove_duplicates_v2(s): return
''.join(dict.fromkeys(s)) # Solution 3: List comprehension with seen tracking def remove_duplicates_v3(s):
seen = set() return ''.join([char for char in s if not (char in seen or seen.add(char))]) # Test
print(remove_duplicates("programming")) # "progamin" print(remove_duplicates("hello")) # "helo"
print(remove_duplicates("aabbcc")) # "abc"
```

Q7: String Compression

Problem: Compress a string using counts of repeated characters. **Example:** Input: "aaabbccccc" Output: "a3b2c4"
Input: "abc" Output: "abc" (compressed version is not shorter)

```
# Solution 1: Iterate and count
def compress_string(s):
    if not s:
        return s
    compressed = []
    count = 1
    for i in range(1, len(s)):
        if s[i] == s[i-1]:
            count += 1
        else:
            compressed.append(s[i-1] + str(count))
            count = 1
    # Add last character
    compressed.append(s[-1] + str(count))
    result = ''.join(compressed)
    return result if len(result) < len(s) else s

# Solution 2: Using groupby from itertools
import groupby
def compress_string_v2(s):
    compressed = ''.join(char + str(len(list(group))) for char, group in groupby(s))
    return compressed if len(compressed) < len(s) else s

# Test
print(compress_string("aaabbccccc")) # "a3b2c4"
print(compress_string("abc")) # "abc"
print(compress_string("aabccccaaa")) # "a2b1c5a3"
```

Q8-Q15: More String Problems (Quick Solutions)

```
# Q8: Check if string contains only digits
def is_all_digits(s):
    return s.isdigit()
# Alternative: return all(c.isdigit() for c in s)

# Q9: Capitalize first letter of each word
def title_case(s):
    return s.title()
# Manual: return ' '.join(word.capitalize() for word in s.split())

# Q10: Find longest word in sentence
def longest_word(s):
    words = s.split()
    return max(words, key=len)

# Q11: Count words in string
def count_words(s):
    return len(s.split())

# Q12: Replace spaces with underscores
def replace_spaces(s):
    return s.replace(' ', '_')

# Q13: Check if substring exists
def contains_substring(s, sub):
    return sub in s

# Q14: Remove all whitespace
def remove_whitespace(s):
    return ''.join(s.split())

# Q15: Convert string to list of characters
def string_to_list(s):
    return list(s)
# Alternative: return [char for char in s]

# Tests
print(is_all_digits("12345")) # True
print(title_case("hello world")) # "Hello World"
print(longest_word("I love Python")) # "Python"
print(count_words("Hello world")) # 2
```

2. LISTS & ARRAYS (12 Questions)

Q16: Find Maximum in List

Problem: Find the maximum element in a list without using max(). **Example:** Input: [3, 7, 2, 9, 1] Output: 9

```
# Solution 1: Iterate through list
def find_max(lst):
    if not lst:
        return None
    max_val = lst[0]
    for num in lst[1:]:
        if num > max_val:
            max_val = num
    return max_val
# Solution 2: Using reduce from functools
import reduce
def find_max_v2(lst):
    return reduce(lambda x, y: x if x > y else y, lst)
# Solution 3: Recursive approach
def find_max_v3(lst):
    if len(lst) == 1:
        return lst[0]
    return max(lst[0], find_max_v3(lst[1:]))
# Test
print(find_max([3, 7, 2, 9, 1])) # 9
print(find_max([5])) # 5
print(find_max([])) # None
```

Q17: Remove Duplicates from List

Problem: Remove duplicates from a list while preserving order. **Example:** Input: [1, 2, 2, 3, 4, 4, 5] Output: [1, 2, 3, 4, 5]

```
# Solution 1: Using set to track seen elements
def remove_duplicates(lst):
    seen = set()
    result = []
    for item in lst:
        if item not in seen:
            seen.add(item)
            result.append(item)
    return result
# Solution 2: Using dict.fromkeys (Python 3.7+)
def remove_duplicates_v2(lst):
    return list(dict.fromkeys(lst))
# Solution 3: List comprehension with tracking
def remove_duplicates_v3(lst):
    seen = set()
    return [x for x in lst if not (x in seen or seen.add(x))]
# Test
print(remove_duplicates([1, 2, 2, 3, 4, 4, 5])) # [1, 2, 3, 4, 5]
print(remove_duplicates([1, 1, 1])) # [1]
```

Q18: Find Second Largest Number

Problem: Find the second largest number in a list. **Example:** Input: [3, 7, 2, 9, 1] Output: 7

```
# Solution 1: Sort and get second last (simple but O(n log n))
def second_largest(lst):
    if len(lst) < 2:
        return None
    unique = list(set(lst)) # Remove duplicates
    unique.sort()
    return unique[-2]
if len(unique) >= 2 else None # Solution 2: Two variables (O(n), more efficient)
def second_largest_v2(lst):
    if len(lst) < 2:
        return None
    first = second = float('-inf')
    for num in lst:
        if num > first:
            second = first
            first = num
        elif num > second and num != first:
            second = num
    return second
if second != float('-inf') else None # Solution 3: Using sorted unique values
def second_largest_v3(lst):
    unique = sorted(set(lst), reverse=True)
    return unique[1] if len(unique) >= 2 else None
# Test
print(second_largest([3, 7, 2, 9, 1])) # 7
print(second_largest([5, 5, 5])) # None
print(second_largest([10, 20, 20, 30])) # 20
```

Q19: Rotate List/Array

Problem: Rotate a list to the right by k positions. **Example:** Input: [1, 2, 3, 4, 5], k=2 Output: [4, 5, 1, 2, 3]

```
# Solution 1: Using slicing (Most Pythonic)
def rotate_list(lst, k):
    if not lst:
        return lst
    k = k % len(lst) # Handle k > len(lst)
    return lst[-k:] + lst[:-k]
# Solution 2: Using collections.deque from collections
import deque
def rotate_list_v2(lst, k):
    d = deque(lst)
    d.rotate(k)
    return list(d)
# Solution 3: Manual rotation
def rotate_list_v3(lst, k):
    k = k % len(lst)
    return lst[len(lst)-k:] + lst[:len(lst)-k]
# Test
print(rotate_list([1, 2, 3, 4, 5], 2)) # [4, 5, 1, 2, 3]
print(rotate_list([1, 2, 3], 1)) # [3, 1, 2]
print(rotate_list([1, 2, 3, 4], 0)) # [1, 2, 3, 4]
```


Q20: Merge Two Sorted Lists

Problem: Merge two sorted lists into one sorted list. **Example:** Input: [1, 3, 5], [2, 4, 6] Output: [1, 2, 3, 4, 5, 6]

```
# Solution 1: Two pointers (O(n+m))
def merge_sorted_lists(lst1, lst2):
    result = []
    i, j = 0, 0
    while i < len(lst1) and j < len(lst2):
        if lst1[i] <= lst2[j]:
            result.append(lst1[i])
            i += 1
        else:
            result.append(lst2[j])
            j += 1
    # Add remaining elements
    result.extend(lst1[i:])
    result.extend(lst2[j:])
    return result

# Solution 2: Combine and sort (simpler but O((n+m)log(n+m)))
def merge_sorted_lists_v2(lst1, lst2):
    return sorted(lst1 + lst2)

# Solution 3: Using heapq.merge
import heapq
def merge_sorted_lists_v3(lst1, lst2):
    return list(heapq.merge(lst1, lst2))

# Test
print(merge_sorted_lists([1, 3, 5], [2, 4, 6])) # [1, 2, 3, 4, 5, 6]
print(merge_sorted_lists_v2([1, 2], [3, 4])) # [1, 2, 3, 4]
print(merge_sorted_lists_v3([1, 2], [3, 4])) # [1, 2, 3, 4]
```

Q21-Q27: More List Problems

```
# Q21: Check if list is sorted
def is_sorted(lst):
    return all(lst[i] <= lst[i+1] for i in range(len(lst)-1))

# Q22: Find missing number in range
def find_missing(lst):
    # Given [1,2,4,5], find missing 3
    n = len(lst) + 1
    expected_sum = n * (n + 1) // 2
    return expected_sum - sum(lst)

# Q23: Flatten nested list
def flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list):
            result.extend(flatten(item))
        else:
            result.append(item)
    return result

# Q24: Chunk list into parts
def chunk_list(lst, size):
    return [lst[i:i+size] for i in range(0, len(lst), size)]

# Q25: Find pairs that sum to target
def find_pairs_sum(lst, target):
    seen = set()
    pairs = []
    for num in lst:
        complement = target - num
        if complement in seen:
            pairs.append((complement, num))
        seen.add(num)
    return pairs

# Q26: Remove all occurrences of value
def remove_all(lst, val):
    return [x for x in lst if x != val]

# Q27: Count frequency of each element
def count_frequency(lst):
    from collections import Counter
    return dict(Counter(lst))

# Tests
print(is_sorted([1, 2, 3, 4])) # True
print(find_missing([1, 2, 4, 5])) # 3
print(flatten([1, [2, 3], [4, [5]]])) # [1, 2, 3, 4, 5]
print(chunk_list([1,2,3,4,5,6], 2)) # [[1,2], [3,4], [5,6]]
print(find_pairs_sum([2,7,11,15], 9)) # [(2, 7)]
```

3. DICTIONARIES & SETS (10 Questions)

Q28: Merge Two Dictionaries

Problem: Merge two dictionaries. If keys overlap, values from second dict should win. **Example:** Input: {'a': 1, 'b': 2}, {'b': 3, 'c': 4} Output: {'a': 1, 'b': 3, 'c': 4}

```
# Solution 1: Using update() method
def merge_dicts(dict1, dict2):
    result = dict1.copy()
    result.update(dict2)
    return result
# Solution 2: Using unpacking (Python 3.5+)
def merge_dicts_v2(dict1, dict2):
    return {**dict1, **dict2}
# Solution 3: Using | operator (Python 3.9+)
def merge_dicts_v3(dict1, dict2):
    return dict1 | dict2
# Solution 4: Using dict() constructor
def merge_dicts_v4(dict1, dict2):
    return dict(list(dict1.items()) + list(dict2.items()))
# Test
d1 = {'a': 1, 'b': 2}
d2 = {'b': 3, 'c': 4}
print(merge_dicts(d1, d2)) # {'a': 1, 'b': 3, 'c': 4}
```

Q29: Invert Dictionary (Swap Keys and Values)

Problem: Swap keys and values in a dictionary. **Example:** Input: {'a': 1, 'b': 2, 'c': 3} Output: {1: 'a', 2: 'b', 3: 'c'}

```
# Solution 1: Dictionary comprehension
def invert_dict(d):
    return {v: k for k, v in d.items()}
# Solution 2: Using zip
def invert_dict_v2(d):
    return dict(zip(d.values(), d.keys()))
# Solution 3: Handle duplicate values (keep all keys)
def invert_dict_v3(d):
    inverted = {}
    for k, v in d.items():
        if v not in inverted:
            inverted[v] = []
        inverted[v].append(k)
    return inverted
# Test
d = {'a': 1, 'b': 2, 'c': 3}
print(invert_dict(d)) # {1: 'a', 2: 'b', 3: 'c'}
# With duplicates
d2 = {'a': 1, 'b': 1, 'c': 2}
print(invert_dict_v3(d2)) # {1: ['a', 'b'], 2: ['c']}
```

Q30: Group Anagrams Using Dictionary

Problem: Group words that are anagrams of each other. **Example:** Input: ["eat", "tea", "tan", "ate", "nat", "bat"] Output: [{"eat", "tea", "ate"}, {"tan", "nat"}, {"bat"}]

```
# Solution 1: Using sorted tuple as key
def group_anagrams(words):
    from collections import defaultdict
    anagram_dict = defaultdict(list)
    for word in words:
        # Sort the word and use as key
        key = tuple(sorted(word))
        anagram_dict[key].append(word)
    return list(anagram_dict.values())

# Solution 2: Using Counter as key
from collections import Counter
def group_anagrams_v2(words):
    anagram_dict = {}
    for word in words:
        # Use sorted string as key
        key = ''.join(sorted(word))
        if key not in anagram_dict:
            anagram_dict[key] = []
        anagram_dict[key].append(word)
    return list(anagram_dict.values())

# Test words = ["eat", "tea", "tan", "ate", "nat", "bat"]
print(group_anagrams(words)) # [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
```

Q31-Q37: More Dictionary & Set Problems

```
# Q31: Find most frequent element
def most_frequent(lst):
    from collections import Counter
    return Counter(lst).most_common(1)[0][0]

# Q32: Check if two lists have common elements
def have_common(list1, list2):
    return bool(set(list1) & set(list2))

# Q33: Remove keys from dictionary
def remove_keys(d, keys):
    return {k: v for k, v in d.items() if k not in keys}

# Q34: Get dictionary value with default
def get_with_default(d, key, default=0):
    return d.get(key, default)

# Q35: Sort dictionary by value
def sort_dict_by_value(d):
    return dict(sorted(d.items(), key=lambda x: x[1]))

# Q36: Find intersection of two sets
def set_intersection(set1, set2):
    return set1 & set2

# Q37: Find union of two sets
def set_union(set1, set2):
    return set1 | set2

# Tests
print(most_frequent([1,2,2,3,3,3])) # 3
print(have_common([1,2,3], [3,4,5])) # True
print(remove_keys({'a':1,'b':2,'c':3}, ['b'])) # {'a': 1, 'c': 3}
print(sort_dict_by_value({'a':3,'b':1,'c':2})) # {'b':1,'c':2,'a':3}
```

4. FUNCTIONS & RECURSION (8 Questions)

Q38: Factorial (Recursion)

Problem: Calculate factorial of n using recursion. **Example:** Input: 5 Output: 120 ($5 * 4 * 3 * 2 * 1$)

```
# Solution 1: Recursive
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

# Solution 2: Iterative
def factorial_iterative(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

# Solution 3: Using reduce from functools
import reduce
def factorial_reduce(n):
    return reduce(lambda x, y: x * y, range(1, n + 1), 1)

# Solution 4: Using math module
import math
def factorial_math(n):
    return math.factorial(n)

# Test
print(factorial(5)) # 120
print(factorial(0)) # 1
print(factorial(10)) # 3628800
```

Q39: Fibonacci Sequence

Problem: Generate nth Fibonacci number. **Example:** Sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21... Input: 6 Output: 8

```
# Solution 1: Recursive (simple but slow - O(2^n))
def fibonacci_recursive(n):
    if n <= 1:
        return n
    return fibonacci_recursive(n-1) + fibonacci_recursive(n-2)

# Solution 2: Memoization (cache results)
def fibonacci_memo(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fibonacci_memo(n-1, memo) + fibonacci_memo(n-2, memo)
    return memo[n]

# Solution 3: Iterative (most efficient - O(n))
def fibonacci_iterative(n):
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b

# Solution 4: Using lru_cache decorator from functools
import functools
@functools.lru_cache(maxsize=None)
def fibonacci_cached(n):
    if n <= 1:
        return n
    return fibonacci_cached(n-1) + fibonacci_cached(n-2)

# Test
print(fibonacci_iterative(6)) # 8
print(fibonacci_iterative(10)) # 55
```

Q40-Q45: More Function & Recursion Problems

```
# Q40: Sum of digits (recursive) def sum_digits(n): if n == 0: return 0 return n % 10 + sum_digits(n // 10)
# Q41: Reverse string (recursive) def reverse_recursive(s): if len(s) <= 1: return s return reverse_recursive(s[1:]) + s[0]
# Q42: Power function (recursive) def power(base, exp): if exp == 0: return 1 return base * power(base, exp - 1)
# Q43: Lambda function examples square = lambda x: x ** 2 add = lambda x, y: x + y is_even = lambda x: x % 2 == 0
# Q44: Map, filter, reduce examples numbers = [1, 2, 3, 4, 5] squared = list(map(lambda x: x**2, numbers))
evens = list(filter(lambda x: x % 2 == 0, numbers)) from functools import reduce total = reduce(lambda x, y: x + y, numbers)
# Q45: Decorator example def timer_decorator(func): import time def wrapper(*args, **kwargs): start = time.time() result = func(*args, **kwargs) end = time.time() print(f"{func.__name__} took {end-start:.2f} seconds") return result return wrapper
@timer_decorator def slow_function(): import time time.sleep(1) return "Done"
# Tests print(sum_digits(123)) # 6 print(reverse_recursive("hello")) # "olleh" print(power(2, 3)) # 8 print(squared) # [1, 4, 9, 16, 25]
```

5. FILE I/O & DATA PROCESSING (8 Questions)

Q46: Read and Process CSV File

Problem: Read a CSV file and calculate average of a numeric column. **CSV Content:** name,age,salary Alice,25,50000 Bob,30,60000 Carol,35,70000

```
# Solution 1: Using csv module
import csv
def process_csv(filename):
    total_salary = 0
    count = 0
    with open(filename, 'r') as f:
        reader = csv.DictReader(f)
        for row in reader:
            total_salary += int(row['salary'])
            count += 1
    return total_salary / count if count > 0 else 0

# Solution 2: Using pandas
import pandas as pd
def process_csv_pandas(filename):
    df = pd.read_csv(filename)
    return df['salary'].mean()

# Solution 3: Manual parsing
def process_csv_manual(filename):
    with open(filename, 'r') as f:
        lines = f.readlines()
    header = lines[0].strip().split(',')
    salary_idx = header.index('salary')
    salaries = []
    for line in lines[1:]:
        values = line.strip().split(',')
        salaries.append(int(values[salary_idx]))
    return sum(salaries) / len(salaries)

# Usage
avg = process_csv('employees.csv')
print(f"Average salary: {avg}")
```

Q47-Q53: More File I/O Problems

```
# Q47: Read file line by line
def read_file_lines(filename):
    with open(filename, 'r') as f:
        return [line.strip() for line in f]

# Q48: Count lines in file
def count_lines(filename):
    with open(filename, 'r') as f:
        return sum(1 for line in f)

# Q49: Find word in file
def find_word_in_file(filename, word):
    with open(filename, 'r') as f:
        for line_num, line in enumerate(f, 1):
            if word in line:
                print(f"Found on line {line_num}: {line.strip()}")

# Q50: Write list to file
def write_list_to_file(lst, filename):
    with open(filename, 'w') as f:
        for item in lst:
            f.write(str(item) + '\n')

# Q51: Read JSON file
import json
def read_json(filename):
    with open(filename, 'r') as f:
        return json.load(f)

# Q52: Write JSON file
def write_json(data, filename):
    with open(filename, 'w') as f:
        json.dump(data, f, indent=2)

# Q53: Copy file content
def copy_file(source, destination):
    with open(source, 'r') as src:
        with open(destination, 'w') as dst:
            dst.write(src.read())

# Usage examples
lines = read_file_lines('data.txt')
count = count_lines('data.txt')
find_word_in_file('data.txt', 'Python')
write_list_to_file([1, 2, 3], 'output.txt')
```

6. OBJECT-ORIENTED PROGRAMMING (6 Questions)

Q54: Create a Simple Class

Problem: Create a Person class with name and age attributes, and a method to display info. **Example Usage:** p = Person("Alice", 25) p.display() # Output: "Alice is 25 years old"

```
# Solution: Basic class with methods
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def display(self):
        print(f"{self.name} is {self.age} years old")
    def birthday(self):
        self.age += 1
        print(f"Happy birthday! {self.name} is now {self.age}")
    def __str__(self):
        return f"Person(name={self.name}, age={self.age})"
    def __repr__(self):
        return f"Person('{self.name}', {self.age})"

# Usage
p = Person("Alice", 25)
p.display() # Alice is 25 years old
p.birthday() # Happy birthday! Alice is now 26
print(p) # Person(name=Alice, age=26)
```

Q55-Q59: More OOP Problems

```
# Q55: Inheritance example
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        pass
class Dog(Animal):
    def speak(self):
        return f"{self.name} says Woof!"
class Cat(Animal):
    def speak(self):
        return f"{self.name} says Meow!"

# Q56: Bank Account class
class BankAccount:
    def __init__(self, balance=0):
        self._balance = balance # Protected attribute
    def deposit(self, amount):
        if amount > 0:
            self._balance += amount
            return True
        return False
    def withdraw(self, amount):
        if 0 < amount <= self._balance:
            self._balance -= amount
            return True
        return False
    def get_balance(self):
        return self._balance

# Q57: Static and class methods
class MathOperations:
    pi = 3.14159
    @staticmethod
    def add(x, y):
        return x + y
    @classmethod
    def circle_area(cls, radius):
        return cls.pi * radius ** 2

# Q58: Property decorator
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius
    @property
    def fahrenheit(self):
        return self._celsius * 9/5 + 32
    @fahrenheit.setter
    def fahrenheit(self, value):
        self._celsius = (value - 32) * 5/9

# Q59: Abstract base class from abc
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height
    def area(self):
        return self.width * self.height

# Tests
dog = Dog("Buddy")
print(dog.speak()) # Buddy says Woof!
account = BankAccount(100)
account.deposit(50)
print(account.get_balance()) # 150
```

7. COMMON ALGORITHMS (8 Questions)

Q60: Binary Search

Problem: Implement binary search on a sorted list. **Example:** Input: [1, 3, 5, 7, 9], target=5 Output: 2 (index of 5)

```
# Solution 1: Iterative binary search
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1 # Not found

# Solution 2: Recursive binary search
def binary_search_recursive(arr, target, left=0, right=None):
    if right is None:
        right = len(arr) - 1
    if left > right:
        return -1
    mid = (left + right) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_recursive(arr, target, mid + 1, right)
    else:
        return binary_search_recursive(arr, target, left, mid - 1)

# Test
arr = [1, 3, 5, 7, 9, 11, 13]
print(binary_search(arr, 7)) # 3
print(binary_search(arr, 10)) # -1
```

Q61-Q67: More Algorithm Problems

```
# Q61: Bubble Sort
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

# Q62: Linear Search
def linear_search(arr, target):
    for i, val in enumerate(arr):
        if val == target:
            return i
    return -1

# Q63: Find GCD (Greatest Common Divisor)
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

# Q64: Check if number is prime
def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

# Q65: Generate prime numbers up to n
def sieve_of_eratosthenes(n):
    primes = [True] * (n + 1)
    primes[0] = primes[1] = False
    for i in range(2, int(n**0.5) + 1):
        if primes[i]:
            for j in range(i*i, n + 1, i):
                primes[j] = False
    return [i for i in range(n + 1) if primes[i]]

# Q66: Two Sum problem
def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []

# Q67: Valid Parentheses
def is_valid_parentheses(s):
    stack = []
    mapping = {')': '(', '}': '{', ']': '['}
    for char in s:
        if char in mapping:
            top = stack.pop() if stack else '#'
            if mapping[char] != top:
                return False
        else:
            stack.append(char)
    return not stack

# Tests
print(bubble_sort([64, 34, 25, 12])) # [12, 25, 34, 64]
print(is_prime(17)) # True
print(sieve_of_eratosthenes(20)) # [2, 3, 5, 7, 11, 13, 17, 19]
print(two_sum([2, 7, 11, 15], 9)) # [0, 1]
print(is_valid_parentheses("()[]{}")) # True
```


8. INTERVIEW TIPS & PATTERNS

Common Python Patterns to Remember

```
# List Comprehensions squares = [x**2 for x in range(10)] evens = [x for x in range(10) if x % 2 == 0]
matrix_flat = [item for row in matrix for item in row] # Dictionary Comprehensions square_dict = {x: x**2
for x in range(5)} filtered_dict = {k: v for k, v in d.items() if v > 10} # Generator Expressions (memory
efficient) gen = (x**2 for x in range(1000000)) # Doesn't create list total = sum(x**2 for x in
range(1000)) # Enumerate (get index and value) for i, val in enumerate(['a', 'b', 'c']): print(f"{i}:
{val}") # Zip (combine iterables) names = ['Alice', 'Bob'] ages = [25, 30] for name, age in zip(names,
ages): print(f"{name} is {age}") # defaultdict (auto-initialize) from collections import defaultdict
counts = defaultdict(int) for item in data: counts[item] += 1 # Counter (frequency counting) from
collections import Counter freq = Counter([1, 2, 2, 3, 3, 3]) print(freq.most_common(2)) # [(3, 3), (2,
2)] # any() and all() has_even = any(x % 2 == 0 for x in numbers) all_positive = all(x > 0 for x in
numbers) # sorted() with key words = ['apple', 'pie', 'a', 'cherry'] sorted_by_length = sorted(words,
key=len) sorted_desc = sorted(numbers, reverse=True) # min() and max() with key longest = max(words,
key=len) person_oldest = max(people, key=lambda p: p['age'])
```

Time & Space Complexity Quick Reference

Operation	Time Complexity	Notes
List append	O(1)	Amortized
List insert at index	O(n)	Shifts elements
List pop()	O(1)	From end
List pop(0)	O(n)	From beginning
Dict get/set/delete	O(1)	Average case
Set add/remove/in	O(1)	Average case
List sort	O(n log n)	Timsort algorithm
Binary search	O(log n)	Sorted list required
Linear search	O(n)	Unsorted list

Interview Best Practices

- **Ask clarifying questions:** Input size? Edge cases? Performance requirements?
- **Think out loud:** Explain your approach before coding
- **Start simple:** Get working solution first, optimize later
- **Use meaningful names:** count_vowels better than cv
- **Test with examples:** Walk through your code with sample input
- **Consider edge cases:** Empty input, None, single element, duplicates
- **Discuss trade-offs:** Time vs space, readability vs performance
- **Know your libraries:** collections, itertools, functools
- **Don't:** Jump into coding without planning

- **Don't:** Write code silently
- **Don't:** Ignore edge cases
- **Don't:** Give up if stuck - ask for hints

Final Tips:

- Practice writing code by hand (whiteboard/paper)
 - Review Python built-in functions and standard library
 - Understand when to use list vs dict vs set
 - Practice explaining your code out loud
 - Time yourself (20-30 min per question)
 - Don't memorize solutions - understand patterns
- Good luck with your interview! ■■**